BÁO CÁO THỰC HÀNH

Thực hành An toàn mạng máy tính

Lab X: NỘI DUNG XYZ

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: dd/mm/yyyy

Nhóm 7 – NT101.R12.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
24521664	Phạm Thanh Thiên	Câu 1, câu 2	50%
24521708	Trương Đức Thịnh	Câu 3, câu 5	50%

https://github.com/truongducthinhuit-cloud/NT101.R12.1-ThucHanh

Nội dung thực hành

Nhiệm vụ 2.1:

Code:

```
def F(right, subkey):
    return (right ^ subkey) & 0x0F

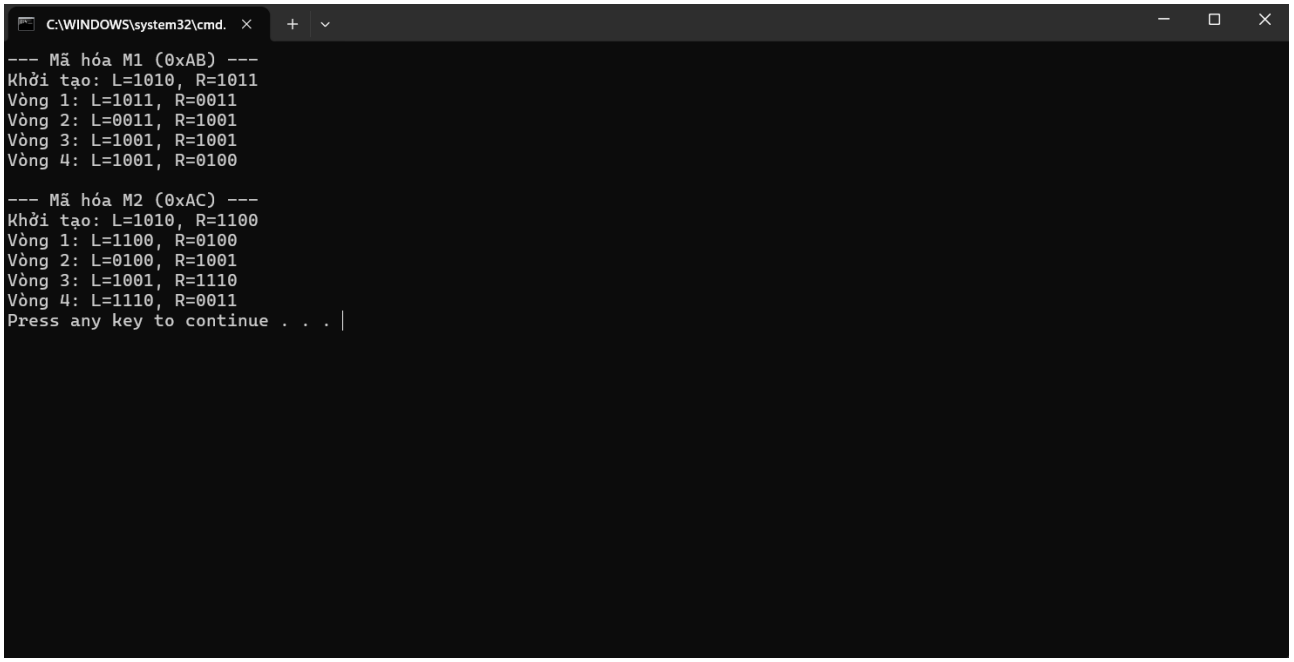
def feistel_round(L_in, R_in, subkey):
    # LE_i = RE_{i-1}
    L_out = R_in
    # RE_i = LE_{i-1} XOR F(RE_{i-1}, K_i)
    R_out = L_in ^ F(R_in, subkey)

    return L_out, R_out

def track_avalanche(msg, key):
    L, R = (msg >> 4) & 0x0F, msg & 0x0F
    subkeys = [key & 0x0F, (key >> 4) & 0x0F, (key + 1) & 0x0F, (key + 2) & 0x0F]

    print(f"Khởi tạo: L={format(L, '04b')}, R={format(R, '04b')}")
    for i in range(4):
        L, R = feistel_round(L, R, subkeys[i])
        print(f"Vòng {i+1}: L={format(L, '04b')}, R={format(R, '04b')}")
    return (L << 4) | R

print("--- Mã hóa M1 (0xAB) ---")
track_avalanche(0xAB, 0x12)
print("\n--- Mã hóa M2 (0xAC) ---")
track_avalanche(0xAC, 0x12)
```



Hình 2.1: Kết quả mô phỏng cấu trúc Feistel và sự lan truyền thay đổi

2.1.1 Giải thích phương pháp thực hiện

Nhiệm vụ này tiến hành mô phỏng hoạt động của cấu trúc mật mã khối Feistel, một cấu trúc nền tảng được ứng dụng trong tiêu chuẩn mã hóa DES. Theo định lý Feistel, khối dữ liệu đầu vào được chia làm hai nửa LE_0 và RE_0, sau đó trải qua các vòng lặp hoán đổi và biến đổi tuần tự. Tại vòng thứ i, phép biến đổi được xác định bởi hai phương trình sau:

- $LE_i = RE_{i-1}$
- $RE_i = LE_{i-1} \oplus F(RE_{i-1}, K_i)$

Trong bài thí nghiệm, hàm giả ngẫu nhiên F được triển khai bằng ngôn ngữ Python theo biểu thức: F(right, subkey) = (right $\oplus$ subkey) & 0x0F. Để quan sát sự lan truyền thay đổi, nhóm đã khởi tạo hai bản rõ đầu vào là $M_1 = 0xAB$ và $M_2 = 0xAC$. Hai giá trị này có điểm đặc biệt là chỉ khác biệt nhau duy nhất một bit ở vị trí cuối cùng (M_1 kết thúc bằng bit 1, trong khi M_2 kết thúc bằng bit 0)

2.1.2. Phân tích kết quả và Nhận xét

Dựa trên kết quả in ra từ chương trình mô phỏng, ta tiến hành đối chiếu chuỗi nhị phân của hai bản rõ qua từng vòng:

- **Trạng thái khởi tạo:**
  - Khối $M_1(0xAB)$: Nửa trái $L_0 = 1010$, Nửa phải $R_0 = 1011$
  - Khối $M_2(0xAC)$: Nửa trái $L_0 = 1010$, Nửa phải $R_0 = 1100$ => Sự khác biệt ban đầu: Khác nhau ở 2 bit cuối của nhánh phải R_0.
- **Sau Vòng 1:**
  - M_1: $L_1 = 1011, R_1 = 0011$
  - M_2: $L_1 = 1100, R_1 = 0100$ => Lúc này, theo phương trình $LE_i = RE_{i-1}$, sự khác biệt từ nửa phải vòng trước đã lan sang nửa trái vòng này. Đồng thời, thông qua hàm nội bộ F kết hợp với khóa con K_1, nửa phải R_1 cũng đã bị biến đổi. Khoảng cách Hamming giữa M_1 và M_2 bắt đầu gia tăng.

- **Sau Vòng 2 và Vòng 3:** Sai khác tiếp tục được truyền chéo giữa nửa trái và nửa phải qua mỗi vòng. Các bit lỗi đóng vai trò là đầu vào mới cho hàm F ở vòng kế tiếp, tạo ra hiệu ứng dây chuyền khuếch đại sự khác biệt.
- **Sau Vòng 4 (Kết quả cuối cùng):**
  - M_1: $L_4 = 1001, R_4 = 0100$
  - M_2: $L_4 = 1110, R_4 = 0011$

    => Chỉ sau 4 vòng lặp ngắn ngủi, chuỗi nhị phân đầu ra của hai bản rõ đã trở nên hoàn toàn khác biệt nhau ở đa số các vị trí bit (Khoảng cách Hamming lớn).

Nhận xét (Kết luận): Thử nghiệm trên đã chứng minh trực quan sự tồn tại và tính hiệu quả của Hiệu ứng thác đổ trong cấu trúc Feistel. Thuộc tính này đảm bảo rằng: khi có một sự thay đổi cực kỳ nhỏ ở đầu vào (dù chỉ là 1 bit của bản rõ hoặc 1 bit của khóa), nó sẽ dẫn đến sự thay đổi to lớn và khó lường ở bản mã đầu ra. Điều này làm cho bản mã trông như một dãy bit ngẫu nhiên, cắt đứt mối liên hệ tuyến tính giữa bản rõ và bản mã, từ đó giúp các hệ mật mã khối (như DES, AES) trở nên an toàn trước những kỹ thuật tấn công phân tích sai biệt

Nhiệm vụ 2.2:

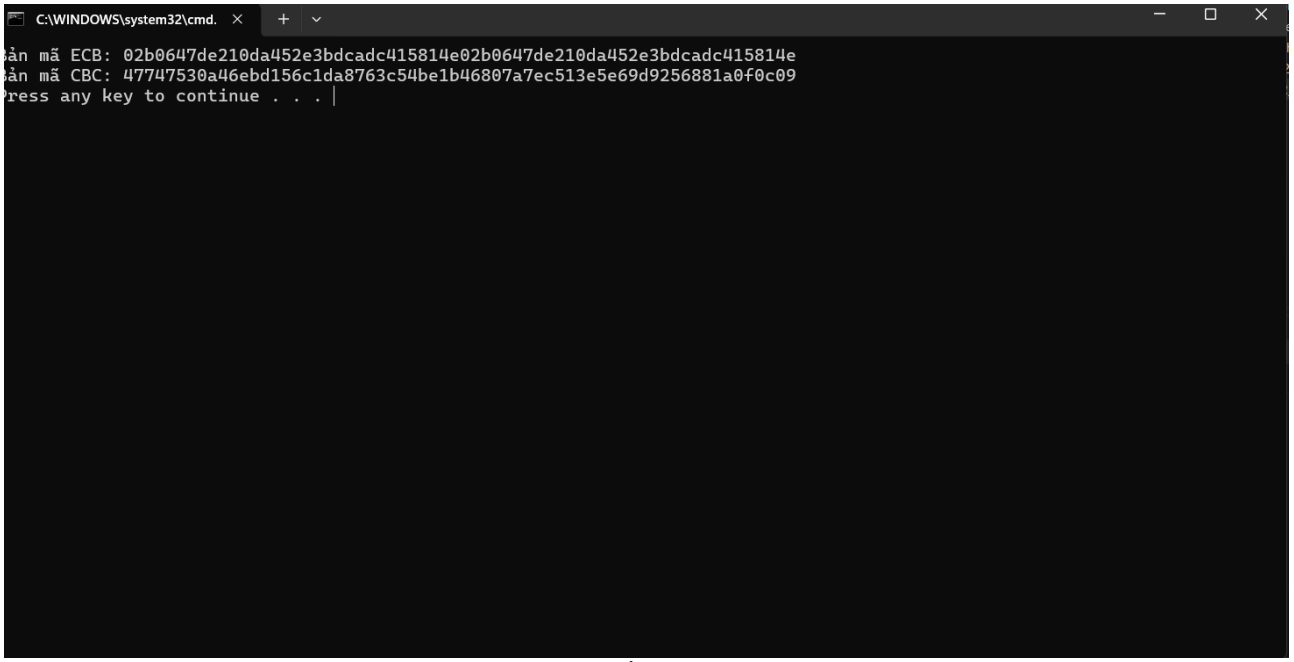
```
from Crypto.Cipher import AES

key = b'1234567890123456'
plaintext = b"UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_"

# 1. AES-ECB
cipher_ecb = AES.new(key, AES.MODE_ECB)
ct_ecb = cipher_ecb.encrypt(plaintext)

# 2. AES-CBC
iv = b'0000000000000000'
cipher_cbc = AES.new(key, AES.MODE_CBC, iv)
ct_cbc = cipher_cbc.encrypt(plaintext)

print(f"Bản mã ECB: {ct_ecb.hex()}")
print(f"Bản mã CBC: {ct_cbc.hex()}")
```



Hình 2.2: Kết quả mã hóa dữ liệu lặp bằng thuật toán AES với chế độ ECB và CBC

2.2.1 Giải thích phương pháp thực hiện

Nhiệm vụ này nhằm đánh giá sự khác biệt về tính bảo mật giữa các chế độ hoạt động của mật mã khối, cụ thể là ECB và CBC

- Dữ liệu đầu vào: Chuỗi ký tự UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_ có độ dài 32 bytes. Đối với thuật toán AES có kích thước khối là 16 bytes, chuỗi này sẽ được chia làm 2 khối dữ liệu đầu vào giống hệt nhau
- Tiến hành mã hóa: Sử dụng cùng một khóa bí mật, hệ thống sẽ mã hóa chuỗi trên bằng hai chế độ ECB và CBC, sau đó in bản mã đầu ra dưới dạng chuỗi Hexadecimal để so sánh.

2.2.3. Phân tích kết quả mã hóa

Bản mã đầu ra gồm 64 ký tự Hex, tương ứng với 32 bytes dữ liệu. Cụ thể như sau:

- Đối với chế độ AES-ECB:
  - Khối mã C1: 02b0647de210da452e3bdcadc415814e
  - Khối mã C2: 02b0647de210da452e3bdcadc415814e

=> Phân tích: Ta thấy C1 hoàn toàn giống hệt C2. Nguyên nhân là do chế độ ECB mã hóa từng khối một cách hoàn toàn độc lập với nhau. Cùng một bản rõ P đưa qua cùng một khóa K sẽ luôn sinh ra cùng một bản mã C. Điều này là một điểm yếu chí mạng vì nó làm lộ các mẫu lặp lại của dữ liệu gốc, giúp kẻ tấn công dễ dàng suy đoán cấu trúc thông tin.

- Đối với chế độ AES-CBC:
  - Khối mã C1: 47747530a46ebd156c1da8763c54be1b

- Khối mã C2: 46807a7ec513e5e69d9256881a0f0c09

=> Phân tích: Mặc dù P1 giống hệt P2, nhưng C1 lại hoàn toàn khác biệt so với C2. Nguyên lý của CBC là sử dụng kết quả bản mã của khối trước (C(i-1) để thực hiện phép XOR với bản rõ của khối hiện tại Pi trước khi đưa vào thuật toán mã hóa. Nhờ cơ chế móc xích này kết hợp với Vector khởi tạo ở khối đầu tiên, các mẫu dữ liệu lặp lại đã bị che giấu hoàn toàn.

2.2.4. Tìm hiểu và nhận xét về các Mode of Operation khác

Ngoài ECB và CBC, NIST (SP 800-38A) còn định nghĩa các chế độ khác như CFB , OFB và CTR . Dựa trên sơ đồ hoạt động, nhóm rút ra nhận xét:

- CFB và OFB: Biến mật mã khối hoạt động tương tự như một mật mã luồng . Dữ liệu mã hóa không phụ thuộc vào kích thước khối cố định, do đó không cần kỹ thuật đệm . Nhờ sử dụng cơ chế phản hồi kết hợp với IV, các khối bản rõ giống nhau cũng sẽ tạo ra các khối bản mã khác nhau.
- CTR: Thay vì dùng bản mã trước đó, CTR mã hóa một bộ đếm tịnh tiến rồi XOR với bản rõ. Chế độ này bảo mật tốt như CBC, không làm lộ cấu trúc lặp lại của dữ liệu, nhưng có ưu điểm vượt trội là cho phép xử lý song song do các khối không phụ thuộc lẫn nhau, giúp tăng tốc độ mã hóa/giải mã.

Kết luận chung: Không bao giờ nên sử dụng chế độ ECB để mã hóa dữ liệu dài hơn một khối do lỗ hổng để lộ cấu trúc dữ liệu. Nên ưu tiên sử dụng CBC hoặc CTR tùy thuộc vào yêu cầu về hiệu suất và tính song song của hệ thống.

Nhiệm vụ 2.3:

- Code

```
from Crypto.Cipher import DES
# Hàm hỗ trợ đếm số bit khác nhau
def count_diff_bits(cipher1, cipher2):
    bin1 = bin(int.from_bytes(cipher1, 'big'))[2:].zfill(64)
    bin2 = bin(int.from_bytes(cipher2, 'big'))[2:].zfill(64)
    return sum(b1 != b2 for b1, b2 in zip(bin1, bin2))
def avalanche_test(key):
    p1 = b'STAYHOME'
    p2 = b'STAYHOMA'
    # TODO 1:
    cipher = DES.new(key, DES.MODE_ECB)
    c1 = cipher.encrypt(p1)
    c2 = cipher.encrypt(p2)
    # TODO 2:
    diff_bits = count_diff_bits(c1, c2)
```

```
    # TODO 3:
    percent = (diff_bits / 64) * 100
    # In kết quả chi tiết phục vụ báo cáo
    print(f"--- Kiểm tra với Key: {key.decode()} ---")
    print(f"  + Bản mã C1 (Hex): {c1.hex()}")
    print(f"  + Bản mã C2 (Hex): {c2.hex()}")
    print(f"  + Số bit khác nhau: {diff_bits} / 64 bits")
    print(f"  + Tỷ lệ phần trăm thay đổi: {percent:.2f}%\n")
avalanche_test(b'87654321')
mssv_list = [
    b'24521708',
    b'24521664'
]
for mssv in mssv_list:
    avalanche_test(mssv)
```

-Testcase:

```
--- Kiểm tra với Key: 87654321 ---
  + Bản mã C1 (Hex): a1c4c1315f860535
  + Bản mã C2 (Hex): ee13eb06301f96be
  + Số bit khác nhau: 37 / 64 bits
  + Tỷ lệ phần trăm thay đổi: 57.81%

--- Kiểm tra với Key: 24521708 ---
  + Bản mã C1 (Hex): 2501bc8016b8fbda
  + Bản mã C2 (Hex): c97acec18effb5ce
  + Số bit khác nhau: 30 / 64 bits
  + Tỷ lệ phần trăm thay đổi: 46.88%

--- Kiểm tra với Key: 24521664 ---
  + Bản mã C1 (Hex): 56bab41aea3f9ff2
  + Bản mã C2 (Hex): 3b7529c8addba5fa
  + Số bit khác nhau: 33 / 64 bits
  + Tỷ lệ phần trăm thay đổi: 51.56%

Press any key to continue . . .
```

- Nhận xét: Dù hai chuỗi bản rõ đầu vào STAYHOME và STAYHOME chỉ khác biệt đúng 1 ký tự cuối cùng (tương đương thay đổi 1 bit ở mức nhị phân), bản mã thu được sau quá trình mã hóa bằng DES hoàn toàn biến đổi và không còn bất kỳ dấu vết tương đồng nào. Ảnh hưởng của việc thay đổi Khóa mã hóa (Key): Khi kiểm tra với khóa mặc định lẫn các khóa là Mã số sinh viên của các thành viên trong nhóm (24521708 và 24521664), tỷ lệ bit thay đổi vẫn duy trì ổn định quanh mức 50%. Điều này cho thấy tính ngẫu nhiên và độ phân tán của thuật toán không phụ thuộc vào giá trị cụ thể của khóa.

Nhiệm vụ 2.5:

- Bảng so sánh

Tiêu chí	DES	3DES (TDEA)	AES
Cấu trúc	Feistel, 16 vòng	Ba lần DES (E-D-E), 48 vòng	SPN, không phải Feistel
Kích thước khối	64 bit	64 bit	128 bit

Độ dài khóa	56 bit hiệu dụng (64 bit gồm 8 bit parity)	168 bit (3 khóa) hoặc 112 bit (2 khóa)	128, 192 hoặc 256 bit
Độ mạnh thực tế	~56 bit, brute-force được	~112 bit (do tấn công meet-in-the-middle)	128 / 192 / 256 bit
Số vòng	16	48	10 / 12 / 14
Hiệu năng	Nhanh hơn 3DES	Chậm (gấp ~3 lần DES)	Nhanh, có hỗ trợ phần cứng (AES-NI)
Trạng thái	Đã bị loại bỏ (NIST rút FIPS 46-3 năm 2005)	NIST không cho phép dùng mã hóa mới sau 2023	Tiêu chuẩn hiện hành (FIPS 197, 2001)

- Giải thích: 2DES mã hóa hai lần với hai khóa độc lập: **C = E_K2(E_K1(P))**.

Nhìn qua thì khóa dài 112 bit, nên brute-force cần 2^112 phép thử. Nhưng với **meet-in-the-middle**, kẻ tấn công chỉ cần biết một cặp (P, C):

1. Với mọi K1 (2^56 giá trị), tính X = E_K1(P) và lưu vào bảng tra.
2. Với mọi K2 (2^56 giá trị), tính Y = D_K2(C) và tra xem Y có trùng X nào trong bảng không.
3. Mỗi lần trùng cho ra một cặp (K1, K2) ứng viên. Vì có khoảng 2^48 ứng viên sai (do khối chỉ 64 bit), cần thêm một hoặc hai cặp (P, C) nữa để lọc.

Chi phí tính toán khoảng 2^57, tức gấp đôi DES chứ không phải 2^56 lần. Đổi lại cần bộ nhớ lớn (~2^56 mục). Kết luận là 2DES gấp đôi số lần mã hóa nhưng chỉ tăng thêm khoảng 1 bit độ mạnh, nên không đáng dùng.

Vì vậy người ta nhảy thẳng lên 3DES. Với ba lần mã hóa, tấn công meet-in-the-middle vẫn cần khoảng 2^112, đủ mạnh vào thời điểm đó. Đây cũng là lý do 3DES dùng dạng E-D-E: khi K1 = K2 = K3, nó tương thích ngược với DES thường.

- Minh chứng đo giá trị


```
import time
from Crypto.Cipher import DES, DES3, AES
from Crypto.Random import get_random_bytes

data = get_random_bytes(10 * 1024 * 1024)  # 10 MB (bội của 8 và 16)

def bench(name, cipher):
    t = time.perf_counter()
    cipher.encrypt(data)
    print(f"{name}: {time.perf_counter() - t:.3f} s")

bench("DES     ", DES.new(get_random_bytes(8), DES.MODE_ECB))
bench("3DES    ", DES3.new(DES3.adjust_key_parity(get_random_bytes(24)),
DES3.MODE_ECB))
bench("AES-128 ", AES.new(get_random_bytes(16), AES.MODE_ECB))
bench("AES-256 ", AES.new(get_random_bytes(32), AES.MODE_ECB))
```

- Kết quả

